

Αρχές Γλωσσών Προγραμματισμού & Μεταφραστών

Τμήμα Μηχανικών Η/Υ & Πληροφορικής — Πανεπιστήμιο Πατρών
Εαρινό Εξάμηνο 2026

Εργαστηριακή Άσκηση — Λεξικός & Συντακτικός Αναλυτής `mysqlq` (Flex / Bison)

Ονοματεπώνυμο	ΑΜ	Έτος	e-mail
Κάνδηλας Γεώργιος	1115510	2ο	up1115510@ac.upatras.gr
Καρπέτας Απόστολος	1115507	2ο	up1115507@ac.upatras.gr

1. Εισαγωγή

Η εργασία υλοποιεί έναν λεξικό και συντακτικό αναλυτή για την ψευδογλώσσα `mysqlq` χρησιμοποιώντας Flex (lexer) και Bison (parser). Χωρίζεται σε τρία ερωτήματα: το Q1 ελέγχει συντακτική ορθότητα, το Q2 προσθέτει σημασιολογική ανάλυση (symbol table, type checking), και το Q3 επεκτείνει με JOIN, AS aliases και qualified αναφορές.

Το πρόγραμμα καλείται ως `myParser.exe file_name`. Σε επιτυχία εκτυπώνει το αρχείο και μήνυμα επιτυχίας· σε αποτυχία, εκτυπώνει τις γραμμές μέχρι το σφάλμα και διαγνωστικό μήνυμα με αριθμό γραμμής.

2. Γραμματική BNF

Σύμβολα: κεφαλαία = τερματικά (tokens), πεζά = μη-τερματικά, ϵ = κενή παραγωγή.

2.1 Βάση (κοινή για Q1 / Q2)

```
stmt_list →  $\epsilon$  | stmt_list stmt

stmt → create_stmt | select_stmt

create_stmt → CREATE TABLE IDENT ( col_def_list ) ;

col_def_list → col_def | col_def_list , col_def

col_def → IDENT INT
| IDENT FLOAT
| IDENT VARCHAR ( INT_LIT )

col_expr_list → * | col_ref_list
```

`col_ref_list` $\rightarrow$ `col_ref` | `col_ref_list` , `col_ref`

`opt_where` $\rightarrow$ ϵ | `WHERE condition`

`condition` $\rightarrow$ `condition OR condition`

| `condition AND condition`

| `NOT condition`

| `( condition )`

| `col_ref comp_op literal`

| `col_ref IN ( in_literal_list )`

| `col_ref NOT IN ( in_literal_list )`

`comp_op` $\rightarrow$ `=` | `!=` | `>` | `<` | `>=` | `<=`

`literal` $\rightarrow$ `INT_LIT` | `FLOAT_LIT` | `STR_LIT`

`in_literal_list` $\rightarrow$ `literal` | `in_literal_list` , `literal`

`opt_group` $\rightarrow$ ϵ | `GROUP BY col_ref_list`

`opt_order` $\rightarrow$ ϵ | `ORDER BY col_ref_list`

`opt_limit` $\rightarrow$ ϵ | `LIMIT INT_LIT`

2.2 Q1

`select_stmt` $\rightarrow$ `SELECT col_expr_list FROM IDENT`

`opt_where opt_group opt_order opt_limit ;`

`col_ref` $\rightarrow$ `IDENT` | `IDENT . IDENT`

2.3 Q2

`select_stmt` $\rightarrow$ `SELECT select_col_ref_list FROM IDENT`

`opt_where opt_group opt_order opt_limit ;`

| `SELECT * FROM IDENT`

`opt_where opt_group opt_order opt_limit ;`

```
select_col_ref_list → col_ref | select_col_ref_list , col_ref
```

```
col_ref → IDENT -- χωρίς qualified ref (δεν υπάρχουν JOIN/alias στο Q2)
```

2.4 Q3 — επεκτάσεις

```
select_stmt → SELECT col_expr_list FROM table_ref join_list
```

```
opt_where opt_group opt_order opt_limit ;
```

```
table_ref → IDENT | IDENT AS IDENT
```

```
join_list → ε
```

```
| join_list JOIN table_ref ON qualified_col_ref = qualified_col_ref
```

```
qualified_col_ref → IDENT . IDENT
```

```
col_ref → IDENT | IDENT . IDENT
```

3. Λεξικός Αναλυτής (Flex)

Τα keywords είναι case-insensitive· τα identifiers είναι case-sensitive. Whitespace και σχόλια (-- ... και /* ... */) αγνοούνται. Τα block comments χειρίζονται με regex που αποφεύγει false matches σε multiline.

- IDENT: `[a-zA-Z_][a-zA-Z0-9_]*` (εφόσον δεν ταιριάζει keyword)
- INT_LIT: `-?[0-9]+`
- FLOAT_LIT: `-?[0-9]+\.[0-9]+`
- STR_LIT: `'[^']*'` — αφαιρούνται τα single quotes στην τιμή
- Keywords: SELECT FROM WHERE GROUP ORDER BY LIMIT CREATE TABLE JOIN ON AS IN AND OR NOT INT FLOAT VARCHAR
- Operators: `= != > < >= <=`
- Punctuation: `, . ; ( ) *`

4. Ερώτημα 1 — Συντακτική Ανάλυση

Ο parser του Q1 ελέγχει μόνο τη συντακτική ορθότητα — δεν τηρεί symbol table. Επαληθεύει ότι κάθε statement ακολουθεί τη γραμματική και οι παρενθέσεις είναι ζευγοποιημένες.

- Αρχικό σύμβολο stmt_list: επιτρέπει κενό αρχείο (μέσω ε) και οποιοδήποτε πλήθος statements.

- Προτεραιότητα τελεστών: %left TOK_OR / %left TOK_AND / %right TOK_NOT ώστε η `a AND b OR c` παρσάρεται ως `(a AND b) OR c`.
- Εκτύπωση σφάλματος: ο `main()` διαβάζει το αρχείο σε `buffer`, κάνει `rewind()` και ξεκινά `parsing`. Η `yyperror()` εκτυπώνει τις γραμμές μέχρι το σφάλμα και τον αριθμό γραμμής.
- Αυστηρή σειρά `SELECT`: η γραμματική επιβάλλει `select` → `from` → `where` → `group by` → `order by` → `limit`. Διαφορετική σειρά είναι σφάλμα.

5. Ερώτημα 2 — Σημασιολογική Ανάλυση

Το Q2 επεκτείνει τον parser με `symbol table` και σημασιολογικούς ελέγχους υλοποιημένους αποκλειστικά σε C.

Για κάθε πίνακα αποθηκεύεται το όνομά του και η λίστα στηλών με τον τύπο τους (`enum ColType {TYPE_INT, TYPE_FLOAT, TYPE_VARCHAR, TYPE_UNKNOWN}`).

Υλοποιούμενοι έλεγχοι:

- (a) Μοναδικότητα ονόματος πίνακα σε κάθε `CREATE TABLE`.
- (b) Ο πίνακας στο `FROM` πρέπει να έχει οριστεί με `CREATE`.
- (c) Οι στήλες κάθε πίνακα πρέπει να είναι μοναδικές· διαφορετικοί πίνακες μπορούν να μοιράζονται ονόματα στηλών.
- (d) Στήλες σε `SELECT`, `WHERE`, `GROUP BY`, `ORDER BY` πρέπει να υπάρχουν στον πίνακα του `FROM`.
- (e) Type checking: `INT` ↔ `INT_LIT`, `FLOAT` ↔ `INT_LIT` ή `FLOAT_LIT` (widening), `VARCHAR` ↔ `STR_LIT`. Για `NOT IN`: ο τύπος στήλης πρέπει να είναι συμβατός με όλα τα literals.

Deferred checking `SELECT` στηλών: ο parser βλέπει τις `SELECT` στήλες πριν από το `FROM`. Αποθηκεύονται σε `buffer` και ελέγχονται αφού οριστεί ο active πίνακας.

6. Ερώτημα 3 — JOIN, Aliases, Qualified References

Το Q3 προσθέτει υποστήριξη `JOIN`, `AS aliases` και `qualified αναφορές` πίνακας.στήλη, με τους αντίστοιχους σημασιολογικούς ελέγχους.

- `JOIN`: ο πίνακας πρέπει να υπάρχει στο `symbol table`. Τα ονόματα πινάκων/στηλών στο `ON clause` ελέγχονται.
- `AS alias`: `alias map (alias→table)`. Η `sem_resolve_alias()` αντιστοιχίζει `alias` στο πραγματικό όνομα· αν το όνομα δεν είναι `alias`, επιστρέφεται ως έχει.
- `ON clause`: χρησιμοποιεί ξεχωριστό `non-terminal qualified_col_ref` που δέχεται μόνο `IDENT.IDENT` — αποφεύγει αμφισημία όταν και οι δύο πίνακες έχουν στήλη με το ίδιο όνομα.
- Multiple `JOINS`: λίστα `active tables` αντί για έναν. Κάθε `JOIN` προσθέτει τον πίνακα στη λίστα.

- Ambiguity check: bare στήλη που βρίσκεται σε >1 active tables → warning, απαιτείται qualified reference.
- Reset aliases: τα aliases επαναφέρονται στην αρχή κάθε SELECT statement.

7. Παράδειγμα Εκτέλεσης (Q3)

```
CREATE TABLE Students (student_id INT, gpa FLOAT);

CREATE TABLE Enrollments (enrollment_id INT, student_id INT, grade VARCHAR(2));

SELECT s.student_id, e.grade

FROM Students AS s

JOIN Enrollments AS e ON s.student_id = e.student_id

WHERE s.gpa >= 3.0

AND e.grade NOT IN ('F', 'D');
```

Ακολουθία ελέγχων:

- CREATE TABLE Students → εγγραφή στο symbol table.
- CREATE TABLE Enrollments → εγγραφή στο symbol table.
- FROM Students AS s → sem_from_check("Students") ✓ · εγγραφή alias s→Students.
- JOIN Enrollments AS e ON s.student_id = e.student_id → sem_from_check("Enrollments") ✓ · alias e→Enrollments · ελέγχεται student_id και στους δύο πίνακες.
- SELECT deferred: s.student_id ✓, e.grade ✓.
- WHERE s.gpa >= 3.0 → TYPE_FLOAT vs FLOAT_LIT ✓.
- e.grade NOT IN ('F','D') → TYPE_VARCHAR vs STR_LIT ✓.
- Αποτέλεσμα: εκτύπωση αρχείου + «Syntactic analysis successful.»

8. Screenshots Παραδειγμάτων

Τα screenshots επιτυχούς και ανεπιτυχούς ανάλυσης για κάθε ερώτημα συμπεριλαμβάνονται στο .zip, στον φάκελο screenshots/. Καλύπτονται τουλάχιστον:

- Q1: CREATE TABLE ορθό · CREATE Table με λάθος (έλλειψη ;, λάθος τύπος)
- Q1: SELECT ορθό · SELECT με λάθος σειράς όρων · unmatched parentheses
- Q2: υπαρκτός / ανύπαρκτος πίνακας · ανύπαρκτη στήλη · type mismatch
- Q3: JOIN ορθό · JOIN με ανύπαρκτο πίνακα · ambiguous bare column
- Q3: AS alias — ορθό · χρήση alias σε λάθος πίνακα

9. Σχόλια — Παραδοχές

- Αρνητικά literals (-512, -0.5) αναγνωρίζονται απευθείας από τον lexer.
- Το LIMIT δέχεται μόνο αυστηρά θετικό INT_LIT· μηδέν και αρνητικοί είναι σφάλμα.
- Block comments επιτρέπονται μεταξύ tokens αλλά όχι εντός ενός token (π.χ. `Se/**/lect` → σφάλμα).
- Στο Q2 το col_ref δέχεται μόνο bare IDENT — το table.column επανέρχεται στο Q3.
- Αν χρησιμοποιείται alias για κάποιον πίνακα, όλες οι αναφορές στις στήλες του γίνονται μέσω alias.
- Οι σημασιολογικοί έλεγχοι Q2 & Q3 υλοποιούνται αποκλειστικά σε C.
- FLOAT widening: η σύγκριση `FLOAT_col >= 42` (INT literal) επιτρέπεται· το αντίστροφο (`INT_col >= 3.14`) όχι.